

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	LOKESH S	Reg. No.:	192572151
Section / Batch:	CSE(AI) / 2 ND YEAR	Date:	11/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I LOKESH S (192572151) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: LOKESH S

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Algorithm Design & Analysis — Applied Case Studies

Q1. 0/1 Knapsack – Cloud Resource Allocation

Problem Mapping Each client application i has a memory requirement $w[i]$ (weight) and a revenue value $v[i]$. The cloud provider has a fixed capacity $W = 100$ GB. Since an application is either hosted fully or not at all (no partial allocation), this is exactly the **0/1 Knapsack problem** — select a subset of applications such that total memory ≤ 100 GB and total revenue is maximized.

DP Design Define $dp[i][w]$ = maximum revenue achievable using the first i applications with capacity w .

Recurrence:

$$dp[i][w] = dp[i-1][w] \quad \text{if } weight[i] > w$$
$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-weight[i]] + value[i]) \quad \text{otherwise}$$

Base case: $dp[0][w] = 0$ for all w . Final answer: $dp[n][W]$. Backtracking through the table recovers which applications were selected.

Why DP (justification)

- **Optimal substructure:** the optimal allocation for capacity w using i items depends only on optimal solutions to smaller subproblems ($i-1$ items, reduced capacity).
- **Overlapping subproblems:** a greedy (highest revenue/GB ratio first) approach fails for 0/1 knapsack because items can't be split — greedy can get trapped in a suboptimal combination. DP guarantees the global optimum by exhaustively (but efficiently) exploring combinations via memoized subproblems, since the same (i, w) states recur across different decision paths.

Tools/Platforms: Python (NumPy for table ops), Java/C++ for production allocators, or embedding the DP as a microservice in a Kubernetes-based cloud scheduler; OR-Tools (Google) also has a built-in knapsack solver for large-scale deployment.

Complexity

- Time: $O(n \cdot W)$ — pseudo-polynomial, here $n \times 100$.
- Space: $O(n \cdot W)$, reducible to $O(W)$ using a 1D rolling array since each row only depends on the previous row.

Industry Interpretation: With small n (dozens of tenants) and $W=100$, this runs in milliseconds — fully practical for real-time admission control in a cloud scheduler. It converts a resource-allocation policy into a mathematically optimal, auditable decision rather than heuristic bin-packing, directly maximizing SLA-linked revenue.

Q2. TSP – Held-Karp for Delivery Routing

Problem Mapping The delivery company must visit 15 cities exactly once, returning to the depot, minimizing total travel cost — the classic Traveling Salesman Problem. Brute force is $O(n!)$; **Held-Karp** reduces this to $O(n^2 \cdot 2^n)$ using DP over subsets.

State Representation $dp[S][j]$ = minimum cost to visit all cities in subset S (which includes the depot), ending at city j .

- S is a bitmask representing the subset of visited cities ($2^{15} = 32768$ possible subsets for 15 cities).
- j is the current terminal city.

Recurrence

$dp[\{0\}][0] = 0$ (start at depot, city 0)

$dp[S][j] = \min_{k \in S, k \neq j} \{ dp[S - \{j\}][k] + \text{cost}(k, j) \}$

Final answer: $\min_j \{ dp[\text{FullSet}][j] + \text{cost}(j, 0) \}$ (return to depot).

Feasibility in Real-World Logistics

- For exactly 15 cities, $15^2 \times 2^{15} \approx 7.4\text{M}$ operations — feasible for offline route planning (e.g., nightly route computation).
- Not feasible for real-time re-routing with dozens/hundreds of stops (exponential blowup) — in practice, logistics companies use Held-Karp only for small, fixed clusters and switch to heuristics/metaheuristics (Nearest Neighbor, 2-opt, Genetic Algorithms, Lin-Kernighan, or ILP solvers with time limits) for larger fleets.

Tools: Python (intercools, NumPy bitmask DP), Google OR-Tools (Routing Library — industry standard for VRP/TSP), Gurobi/CPLEX for exact MILP formulations at scale.

Complexity

- Time: $O(n^2 \cdot 2^n)$ — exponential.
- Space: $O(n \cdot 2^n)$ for the DP table.
- At $n=15$: manageable ($\sim 500\text{K}$ states $\times 15$). At $n=25$: already computationally prohibitive ($25^2 \times 2^{25} \approx 2 \times 10^{10}$).

Interpretation: Held-Karp is exact and gives a provably optimal tour but scales poorly — useful as a benchmark/validation tool or for small depots/sub-clusters, while large fleets rely on approximation algorithms trading a small optimality gap for tractable runtime.

Q3. Floyd–Warshall – Telecom Router Shortest Paths

Problem Mapping The telecom network is a weighted graph where routers are nodes and link costs (latency/bandwidth cost) are edge weights. The company needs **all-pairs shortest paths** — Floyd–Warshall is ideal since it computes shortest paths between every router pair simultaneously.

Design — Role of Intermediate Nodes $\text{dist}[i][j]$ is iteratively refined by considering each router k as a potential intermediate hop:

for k in $1..n$:

 for i in $1..n$:

 for j in $1..n$:

$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$

After the k -th iteration, $\text{dist}[i][j]$ reflects the shortest path using only routers $\{1, \dots, k\}$ as intermediates. By the end, all router pairs use the globally optimal path allowing any intermediate hop — directly modeling how a packet may be relayed through multiple routers.

Tools/Technologies: Implemented in network simulators (NS-3, GNS3), routing control planes (SDN controllers using Python/C++ with libraries like NetworkX for prototyping), or embedded in telecom OSS/BSS systems for offline route-table precomputation.

Complexity

- Time: $O(n^3)$ — for n routers.
- Space: $O(n^2)$ for the distance matrix (plus optional predecessor matrix for path reconstruction).

Interpretation: For moderately sized networks (hundreds of routers), $O(n^3)$ is tractable for periodic (not per-packet) route table computation. It gives complete pairwise routing information in one pass — useful for network planning, redundancy analysis, and detecting bottlenecks — though for very large or highly dynamic networks (thousands of nodes, frequent topology changes), incremental or distributed protocols (OSPF/BGP using Dijkstra per node) are preferred over recomputation from scratch.

Q4. Optimal Binary Search Tree – Database Query Optimization

Problem Mapping Database records are keys with known access (query) probabilities. An OBST arranges keys into a BST such that **expected search cost** (weighted by access probability $\times$ depth) is minimized — frequently accessed records sit closer to the root.

Cost Function For keys $k_1 \dots k_n$ with probabilities $p_1 \dots p_n$, cost of a tree is:

$$\text{Cost} = \sum p_i \times (\text{depth}(k_i) + 1)$$

$e[i][j]$ = minimum expected cost of a BST containing keys $k_i \dots k_j$.

Recurrence

$$e[i][i-1] = 0$$

$$w[i][j] = w[i][j-1] + p[j] \quad (\text{cumulative probability weight})$$

$$e[i][j] = \min_{r \in [i,j]} \{ e[i][r-1] + e[r+1][j] + w[i][j] \}$$

Here r is tried as the root of the subtree spanning keys $i..j$; the min is taken over all possible roots.

Influence of Probabilities Keys with higher access probability contribute more to $w[i][j]$, so the DP favors placing them at shallower depths (closer to the root), directly minimizing average search comparisons — unlike a plain balanced BST, which ignores access frequency.

Tools: Implementable in C++/Java as part of a custom indexing layer; concept applies to index structures in DBMS internals (e.g., informing B-tree/B+-tree reorganization strategies), and can be prototyped in Python for index cost analysis.

Complexity

- Time: $O(n^3)$ (naive), improvable to $O(n^2)$ with Knuth's optimization (root monotonicity).
- Space: $O(n^2)$ for the cost/weight tables.

Interpretation: For a database with a static, well-characterized query distribution, OBST yields a provably minimal expected number of comparisons per lookup — translating to lower average query latency. In practice, DBMS engines approximate this idea via frequency-aware index tuning and caching (e.g., hot data closer to root/cache) rather than rebuilding an exact OBST per query pattern, since access probabilities shift over time.

Q5. Dijkstra – Real-Time Navigation

Problem Mapping The navigation system's road network is a weighted graph (nodes = intersections, edge weights = distance/time). Dijkstra's algorithm computes the shortest path from a single source (current location) to a destination (or to all nodes), which fits real-time route computation for non-negative edge weights (distances/times can't be negative).

Step-by-Step Execution

1. Initialize $\text{dist}[\text{source}] = 0$, all others $= \infty$; use a min-priority queue keyed by distance.
2. Extract the node u with minimum tentative distance.
3. For each neighbor v of u : if $\text{dist}[u] + \text{weight}(u,v) < \text{dist}[v]$, update $\text{dist}[v]$ and push into the priority queue (relaxation).
4. Mark u as finalized (visited).
5. Repeat until the destination is finalized or the queue is empty.
6. Reconstruct path via stored predecessor pointers.

Priority Queue Role: Ensures the next node processed is always the closest unvisited node, guaranteeing that once a node is finalized its shortest distance is correct — this is what gives Dijkstra its greedy-but-optimal correctness (for non-negative weights).

Data Structures/Tools

- **Binary heap** (standard priority queue) — $O((V+E) \log V)$.
- **Fibonacci heap** — theoretically $O(E + V \log V)$, rarely used in practice due to constant-factor overhead.
- Implementation: `heapq` in Python, `priority_queue` in C++ STL, or production routing engines like **GraphHopper**, **OSRM (Open Source Routing Machine)**, and **Google Maps' internal routing stack** (which layer contraction hierarchies / A* on top of Dijkstra's core idea).

Complexity

- Adjacency matrix + array: $O(V^2)$.
- Adjacency list + binary heap: $O((V+E) \log V)$.
- Adjacency list + Fibonacci heap: $O(E + V \log V)$.

Interpretation: For city-scale graphs (millions of nodes), plain Dijkstra is too slow for real-time queries, so real navigation systems use A* (Dijkstra + heuristic, e.g., straight-line distance) or precomputed speedups like **Contraction Hierarchies** to get near-instant queries. Dijkstra remains the conceptual foundation and is directly usable for smaller regional graphs or as a fallback/verification method.

Q6. Huffman Coding – Video Streaming Compression

Problem Mapping Symbols (pixel values, motion vectors, or encoded tokens) appear with different frequencies. Huffman coding builds a **variable-length prefix code** — frequent symbols get shorter codes, rare symbols get longer codes — minimizing total encoded size, which is exactly what's needed before/alongside video transmission to cut bandwidth.

Tree Construction

1. Create a leaf node for each symbol with its frequency; put all nodes in a min-priority queue (by frequency).
2. Repeatedly extract the two lowest-frequency nodes, merge them into a new internal node (frequency = sum), and reinsert.
3. Repeat until one node remains — the root of the Huffman tree.
4. Assign 0/1 to left/right edges; each symbol's code = path from root to its leaf.

Encoding Process: Replace each symbol in the stream with its bit-code from the tree; since the code is a **prefix code** (no code is a prefix of another), the encoded stream can be decoded unambiguously by traversing the tree bit by bit.

Tools: Core to compression libraries — zlib/DEFLATE, JPEG's entropy coding stage, H.264/H.265/AV1 video codecs (Huffman/CABAC-based entropy coding), Python's custom implementations via `heapq`, or C++ with `priority_queue` for production-grade encoders (e.g., FFmpeg internals).

Complexity

- Time: $O(n \log n)$ for n distinct symbols (building the priority queue and tree).
- Encoding/decoding a stream of length m : $O(m)$ after the tree is built.

Compression Efficiency: Achieves entropy-optimal prefix coding for known symbol frequencies (approaches the Shannon entropy bound), though it must be paired with other techniques (motion compensation, transform coding, arithmetic coding) in modern video codecs for higher compression ratios.

Interpretation: For a streaming platform, Huffman-style entropy coding directly reduces the bits/second needed to transmit video at a given quality, lowering **CDN bandwidth cost** and enabling smoother playback for bandwidth-constrained users — a foundational building block layered under more sophisticated modern video compression pipelines.

Q7. Minimum Spanning Tree – ISP Network Design

Problem Mapping The ISP needs to connect n cities with cable such that all cities are reachable, using the minimum total cable cost, with no redundant cycles — exactly a **Minimum Spanning Tree** problem on a graph where nodes = cities, edges = possible cable routes with costs.

Kruskal's Algorithm

1. Sort all edges by weight ascending.
2. Iterate through sorted edges; add an edge to the MST if it doesn't form a cycle (checked via **Union-Find/Disjoint Set**).
3. Stop when $n-1$ edges are added.

Prim's Algorithm

1. Start from an arbitrary city; maintain a set of connected cities.
2. Repeatedly add the minimum-weight edge connecting the current set to an unconnected city (using a priority queue/min-heap).
3. Continue until all cities are included.

Comparison

Aspect	Kruskal's	Prim's
Approach	Edge-based, global sort	Vertex-based, greedy expansion
Best for	Sparse graphs	Dense graphs
Data structure	Union-Find (DSU)	Min-heap/priority queue
Time complexity	$O(E \log E)$	$O(E \log V)$ with binary heap

Design Choice: For an ISP connecting cities where not every city-pair has a direct feasible cable route (sparse graph), **Kruskal's** is typically preferred since it works naturally on an edge list without needing full adjacency. If the graph is near-complete (many possible routes), **Prim's** with a heap is more efficient.

Tools/Data Structures: Union-Find with path compression + union by rank (for Kruskal's), binary/Fibonacci heap (for Prim's); implementable via Python (networkx.minimum_spanning_tree), C++ (Boost Graph Library), or geospatial network planning tools (e.g., QGIS with network analysis plugins, ArcGIS Network Analyst) for real-world cable route planning with actual geographic distances/costs.

Complexity

- Kruskal's: $O(E \log E)$ (dominated by sorting) $\approx O(E \log V)$.
- Prim's (binary heap): $O(E \log V)$; with Fibonacci heap: $O(E + V \log V)$.

Interpretation: MST guarantees the cheapest possible fully-connected cable layout with no redundant links — directly minimizing capital expenditure for the ISP. In practice, ISPs often add a few extra edges beyond the strict MST for **redundancy/fault-tolerance** (since a pure tree has no backup path if one cable fails), so MST is used as the cost-optimal baseline that's then selectively augmented for reliability